

Clases Introductorias de MATLAB para Matemática Aplicada.

Preparado por el Ing. O. Guillermo Mena

Introducción.

El presente apunte está dirigido a los alumnos que cursan la materia Matemática Aplicada y esencialmente constituyen una guía sintética de las clases de MATLAB que se dictan al comienzo de cada cursada y que permiten fijar las bases para la utilización de esta herramienta para el desarrollo de las actividades prácticas de la materia.

Como estadísticamente ha sucedido en los últimos cursos, estas clases constituyen el primer vínculo que la mayoría de los alumnos establece con un lenguaje de programación, por lo que el nivel de las mismas apunta básicamente a ese grupo. En los manuales de uso y de referencia que acompañan al programa MATLAB, como en la ventana de ayuda del programa podrá hallarse información adicional sobre el conjunto de sus instrucciones, sus opciones y también varios ejemplos.

Clase 1.

EL MODO INTERACTIVO.

El nombre MATLAB es una abreviatura de MATrix LABoratory.

El paquete MATLAB es un programa para realizar cálculos científicos y de ingeniería, que se encuentra preparado específicamente para realizar cálculos numéricos con matrices. Como caso particular puede también trabajar con números escalares y tiene un lenguaje de programación propio.

En esta primer “Clase” vamos a utilizar el programa en el llamado “modo interactivo”, en el cual ante una instrucción dada por teclado el MATLAB realiza la orden que le impartamos.

Estaremos en este modo interactivo (o ventana de comandos, o “**Command Window**”) cuando en pantalla aparezca el símbolo **>>**

VARIABLES DE MEMORIA Y MANEJO DE MATRICES.

Como la especialidad del programa es el trabajo con matrices, vamos a comenzar dándole instrucciones referidas al manejo de las mismas.

Para cargar los componentes de una matriz en la memoria de la máquina podemos tipear lo siguiente en la denominada “línea de comandos”:

```
>> a=[1 2 3 4 6 4 3 4 5]
```

y, apenas pulsemos la tecla de Retorno (o “**Enter**”) veremos que en la pantalla aparece el resultado de la operación realizada y a partir de este momento

dispondremos en la memoria de la máquina de una matriz fila que tiene como componentes 1 2 3 4 6 4 3 4 5 y con la que podremos trabajar identificándola con la letra **a** con la que hemos bautizado a nuestra primer variable de trabajo. Si queremos sumarle a cada componente de esta matriz fila un valor constante de, por ejemplo, 2 podemos tipear

```
>> b=a+2
```

y nuevamente, apenas pulsemos la tecla de Retorno, aparecerá el resultado de la operación realizada, que nos indica que tenemos otra nueva variable en la memoria de la máquina cuyo nombre es **b** y cuyas componentes son 3 4 5 6 8 6 5 6 7

Si queremos cargar en la memoria de la máquina una matriz de varias filas podemos tipear:

```
>>c=[1 2 0; 2 5 -1;4 10 -1]
```

y veremos que, apenas pulsemos la tecla de Retorno, aparecerá en la pantalla

```
c=
```

```
1 2 0
2 5 -1
4 10 -1
```

mostrándonos que tenemos en la memoria de la máquina una matriz de 3x3 con esas componentes y cuyo nombre es **c**

Como el paquete MATLAB está preparado para trabajar con matrices, nos resultará sumamente sencillo hallar su traspuesta, para lo que debemos tipear la letra identificatoria de la matriz de la que queremos calcular la traspuesta seguida del símbolo apóstrofe (tecla ')

```
>>c'
```

y veremos que, apenas pulsemos la tecla de Retorno, aparecerá en la pantalla

```
ans=
```

```
1 2 4
2 5 10
0 -1 -1
```

mostrándonos que tenemos en la memoria de la máquina una matriz de 3x3 con esas componentes, que es la traspuesta de la matriz **c** y cuyo nombre es **ans** (abreviatura del inglés answer o respuesta), que es el nombre que el MATLAB le asigna a las variables cuando no le hemos estipulado previamente un nombre específico.

Siguiendo los pasos anteriores, en la memoria de la máquina tendremos las siguientes variables:

a, b, c y ans

cada una con un contenido distinto y que podremos “rastrear” con el cursor de desplazamiento vertical de la derecha de la pantalla o escribiendo nuevamente el nombre de la variable que nos interese seguida de la tecla Retorno, por ejemplo si tipeamos

```
>>b
```

al pulsar la tecla de Retorno, aparecerá en la pantalla:

```
b =
```

```
3 4 5 6 8 6 5 6 7
```

mostrándonos el contenido de la variable **b** disponible en la memoria de la máquina.

En cualquier momento podemos saber cuáles son las variables que tenemos almacenadas en la memoria utilizando el comando **who** , si tipeamos

```
>>who
```

aparece en pantalla el listado de los nombres asignados a las variables cargadas hasta ese momento

Vamos ahora a multiplicar dos de las matrices que tenemos en memoria y, como ya se dijo que el programa está específicamente preparado para trabajar con matrices, no tendremos que lidiar con el conjunto de las componentes de la matriz, sino que simplemente nos bastará con recordar los nombres de las variables que contienen la matrices a multiplicar, así, tipeando:

```
>>d=c*ans
```

obtendremos el producto matricial de la matriz **c** por su traspuesta, y el resultado lo tendremos en almacenado en la variable **d** (nótese que ya no volveremos a insistir sobre la necesidad de pulsar la tecla Retorno para que el comando efectivamente se ejecute)

```
d=
```

```
5 12 24
```

```
12 30 59
```

```
24 59 117
```

Se puede utilizar el denominado “operador punto” para multiplicar los elementos de la matriz elemento a elemento y no matricialmente, tipeando lo siguiente :

```
>>e=c.*ans
```

y el resultado será:

```
e=
```

```
1 4 0
```

```
4 25 -10
```

```
0 -10 1
```

Podemos también hallar la inversa con la siguiente instrucción:

```
>>X=inv(c)
```

obteniendo, en forma inmediata:

```
X=
```

```
5 2 -2
```

```
-2 -1 1
```

```
0 -2 1
```

Podemos demostrar (recordar?) que el producto de la inversa por la matriz **c** es la matriz identidad, tipeando:

```
>>I=inv(c)*c
```

También se pueden hallar submatrices, a partir de matrices ya ingresadas en la memoria de la máquina, por ejemplo:

```
>>f=[1 2 3 4; 5 6 7 8;9 0 1 2]
```

```
>>subf=f(1:2,2:3)
```

habremos creado una nueva matriz llamada **subf** que contiene las dos primeras filas de la matriz **f** y la segunda y la tercera columna de la matriz **f**

BUSCANDO APROVECHAR LOS RUDIMENTOS...

Con los pocos conceptos desarrollados, vamos a intentar resolver el siguiente sistema de ecuaciones:

$$\begin{aligned}x - y + 3z &= 1 \\ 2x + 3y - 2z &= 4 \\ 5x + y - 3z &= 6\end{aligned}$$

Lamentablemente no alcanza con tipear el sistema tal cual está indicado en los renglones precedentes, sino que deberemos “adaptarnos” al tipo de instrucciones que el MATLAB es capaz de interpretar. En este caso deberemos almacenar en la memoria de la máquina una matriz que tenga los coeficientes del sistema (a la que podemos darle, por ejemplo, el nombre **coef** o cualquier otro), un vector columna que tenga los términos independientes (al que podemos llamar **tind** o como se nos ocurra) y luego podremos resolver el sistema utilizando la función **inv** ya vista.

Guiándonos estrictamente con los conceptos vistos hasta ahora, podremos lograr la solución del sistema tipeando estas tres líneas:

```
>>coef=[1 -1 3;2 3 -2;5 1 -3]
```

```
>>tind=[1 4 6]'
```

```
>>solucion=inv(coef)*tind
```

y, además de los resultados parciales correspondientes a la carga de las variables en memoria, veremos en pantalla la solución del sistema:

```
solucion=
```

```
1,1667
```

```
0,6667
```

```
0,1667
```

ALGO MÁS SOBRE LAS VARIABLES DE MEMORIA.

En el MATLAB vimos que con distintas letras (o palabras) se puede representar a distintas variables de memoria. No habíamos mencionado que estas palabras no podrán contener espacios y que no resultan indistintas las letras mayúsculas de las minúsculas.

Por ejemplo, si a continuación de las últimas tres líneas de comando tipeamos

```
>>Tind=[2;3;4]
```

en la memoria de la máquina, además de las variables **a, b,c, ans,d, e , X, Y, coef, tind** y **solucion** tendremos también la variable **Tind** y, si damos la siguiente instrucción:

```
>>solucion=inv(coef)*Tind
```

obtendremos la solución de otro sistema de ecuaciones, cuyo resultado será:

```
solucion=
```

```
1,000
```

```
0,7143
```

```
0,5714
```

Además, debe observarse que dado que hemos llamado exactamente igual a las dos variables, ya no estará en la memoria de la máquina la solución del primer sistema de ecuaciones, y el MATLAB sólo almacena el último resultado calculado en la variable **solucion**.

ALMACENAMIENTO DE LA SESION DE TRABAJO

Se puede almacenar en un documento word todos las instrucciones que le hemos dado al programa y las respuestas que obtenemos utilizando la siguiente orden

```
>>diary 'matlab100312.doc'
```

a partir del momento en que demos Enter a esta orden, el Matlab comienza a “almacenar” instrucciones y respuestas, y las grabará en un archivo cuyo nombre coloquemos entre apóstrofes simples (en este caso el nombre dado al archivo será *matlab100312.doc*) . Almacenará todas las instrucciones y resultados dados hasta el momento en que demos la siguiente orden

```
>>diary off
```

ALGUNAS FUNCIONES INCORPORADAS Y SU GRAFICACIÓN.

El paquete MATLAB tiene un conjunto importante de funciones disponibles para su uso , algunas de las cuales son las siguientes:

abs(#) cos(#) exp(#) log(#) log10(#) cosh(#) sin(#) tan(#) sqrt(#) humps(#)

en la línea de comandos se pueden utilizar y combinar estas funciones, como por ejemplo tipeando:

```
>>3*cos(sqrt(4.7))
```

obtendremos

```
ans=
```

```
-1,6869
```

Resultado de obtener el coseno de la raíz cuadrada del número 4,7 y luego multiplicado por el número 3

Podemos también obtener gráficos de estas funciones con instrucción **fplot**, por ejemplo tipeando:

```
>>fplot('cos',[0,pi])
```

obtendremos un gráfico de la función coseno(x) en el intervalo $[0,\pi]$

Podemos también encontrar adónde se hace cero una determinada función (hallar las raíces) utilizando la instrucción **fzero**

Por ejemplo, tipeando:

```
>>z=fzero('humps',1)
```

encuentramos el cero de la función 'humps'(o joroba) comenzando a buscar el cero de la función con el número 1, y mostrándonos el resultado que queda almacenado en la variable de memoria llamada **z**

Para ver cómo es esta función “joroba” podemos tipear

```
>>fplot('humps',[0,2])
```

y al hacerlo veremos la forma de esta función, a la vez que habremos perdido el gráfico anterior donde teníamos dibujada la función coseno. Si queremos seguir graficando sin perder los gráficos anteriores debemos tipear

```
>>hold on
```

mediante este comando queda vigente el último gráfico dibujado de manera tal que el próximo comando gráfico que le demos lo ejecutará sobre el mismo, así tipeando a continuación las siguiente líneas:

```
>>grid
```

```
>>plot(z,0,'co')
```

En el mismo gráfico de la función joroba se trazará una grilla con espaciamientos de 0,5 en abcisas y de 20 en ordenadas, y se dibujará una circunferencia de color celeste en el punto de coordenadas (z, 0) es decir en la raíz hallada.

Podemos también encontrar el mínimo de una función con la instrucción **fmin**, tipeando:

```
>>m=fmin('humps',0.25,1)
```

almacenaremos en la variable de memoria **m** el mínimo de la función joroba en el intervalo [0.25, 1]

Si ahora tipeamos:

```
>>plot(m, humps(m),'ro')
```

Veremos una circunferencia de color rojo en el gráfico que tenemos vigente, mostrándonos la ubicación del mínimo hallado.

Para graficar nuevas funciones “limpiando” el gráfico anterior deberemos tipear previamente al comando de graficación lo siguiente:

```
>>hold off
```

Ya veremos que también se puede graficar en 3 dimensiones, pero por ahora señalemos que hemos aprendido 2 instrucciones de graficación en dos dimensiones muy poderosas:

-fplot: para dibujar una función continua en un determinado intervalo

-plot: para dibujar un punto de coordenadas x,y

C l a s e 2.

PROGRAMAR CON EL MATLAB.

Vimos como trabaja el MATLAB con matrices y como se pueden hacer algunas operaciones sencillas desde la denominada “línea de comandos” y como íbamos recibiendo los resultados en forma inmediata (modo interactivo).

En la medida que lo que pretendamos resolver no implique demasiadas operaciones con el modo interactivo será suficiente.

Sin embargo, vamos a ver que normalmente no son operaciones sencillas las que estamos convocados a realizar para la resolución de los distintos métodos que se desarrollarán a lo largo del cuatrimestre.

Seguramente cada método involucrará un conjunto de operaciones, ejecutadas en una determina secuencia y repetirlo (iterar) hasta que el resultado sea satisfactorio.

Para realizar este tipo de tarea será necesario incorporar una herramienta adicional: la programación.

Vamos a aprender un poco a programar, es decir a encadenar comandos en una secuencia determinada, y a dejarlos agrupados en un nombre único que podremos llamar en otro momento y volver a usar cuantas veces nos resulte necesario.

ARCHIVOS M.

Los programas los vamos a almacenar en archivos especiales, llamados archivos “m” (porque tienen esa extensión), y de a poquito vamos a tener que incorporar algunos términos nuevos que nos permitan enlazar o encadenar estos comandos (lenguaje de programación). Afortunadamente este lenguaje de programación resulta muy similar a muchos otros utilitarios, con pequeñas variantes, así que lo que aprendamos no será de utilidad exclusiva para usar el MATLAB.

Pero aún antes de comenzar, sepamos que los archivos m se utilizan también para almacenar funciones, y para almacenar datos.

Y en realidad vamos a comenzar por aquí, por la creación de archivos que contengan funciones, o sea archivos “m” que contengan funciones.

Para eso vamos a necesitar cargar la función mediante un “editor”, y vamos a poder “pasar argumentos” a esa función, la que podremos usar en la línea de comandos como cualquiera de las pocas funciones que vimos (como fmin, fzero)

Para crear archivos m tendremos que seleccionar del menú desplegable superior las siguientes opciones:

File

New

M-file

Y se despliega el editor de archivos m del Matlab, que es un espacio en el que podremos dar comandos o instrucciones como en el modo interactivo, pero, a diferencia de lo que ocurre en aquel, una vez escrita la orden por más que apretemos la tecla Enter esa orden no se ejecuta

CREACIÓN DE ARCHIVOS DE FUNCIONES

Vamos a crear una función que represente al polinomio $x^4 - 4x + 1$

Para eso primero vamos a tener que ir a un editor (normalmente el mismo programa MATLAB nos deriva a uno) y vamos a tener que respetar cierta sintaxis para que el MATLAB entienda que estamos intentando crear una función.

Como la función a ingresar es bastante sencilla, veremos que el tan (a esta altura) promocionado archivo “m” es también muy sencillo, y consistirá (en este caso especial) de sólo dos renglones:

en el primer renglón deberemos tipear:

function f=poli1(x)

la primer palabra es fija y le indica al MATLAB que se está enfrentando con una función (function), luego viene un espacio y a continuación una letra f, que es la denominación que le vamos a dar internamente al polinomio. Luego tipearemos un signo igual y la palabra poli1, que es el nombre con el cual vamos a grabar a este archivo y que será el nombre con el que “invoquemos” a esta función desde la línea de comandos (...en algún lugar del disco debe haber un archivo que se llame fzero.m)

Luego, y entre paréntesis, colocamos los denominados “argumentos” de la función, en este caso es una función que depende sólo de x así que ese es el argumento (**x**)

Después de este primer renglón, que sería algo así como una “presentación”, en el siguiente renglón debemos introducir la definición de la función.

f=x.^4 - 4.*x + 1

f es la denominación que prometimos darle en el primer renglón

.^ porque pretendemos que por un momento el MATLAB se olvide de su especialidad, que son las matrices, y que no necesitamos (no necesitamos que interprete que x es una matriz, ya que para nosotros será un número, o sea un escalar, por eso usamos la notación .^ (ya mencionado en la Clase 1 como “operador punto”)

. * por la misma razón (para garantizar que no se pretende trabajar con matrices).

Así que ahora sólo nos resta grabar esta función con un determinado nombre, que deberá ser poli1.m

y listo!!!

UTILIZANDO LAS FUNCIONES CREADAS - MÉTODO DE BISECCIÓN.

Volvamos ahora a la línea de comandos del MATLAB para usar la función creada en el párrafo anterior.

Escribiendo:

>>poli1(0)

el MATLAB se pone a trabajar silenciosamente y nos devuelve el resultado del polinomio evaluado en el punto x=0, y nos muestra el resultado en pantalla:

f=1

si escribimos

>>poli1(1)

nos devuelve en pantalla:

ans=-2

Con lo que rápidamente comprobamos que nuestra función ya está “funcionando”.

Como aplicación de lo desarrollado hasta aquí, vamos a usar la función creada para la resolución del problema de encontrar las raíces de una función utilizando el método de Bisección.

Vamos a aproximar un cero de esta función, utilizando, en el intervalo (0, 0,5) , el método de Bisección.

La secuencia de instrucciones a utilizar para la resolución es la siguiente:

>>a=0;

>>poli1(a)

era 1

>>b=0.5;

>>poli1(b)

f=-0.9375

Vemos que como son de distinto signo podemos aplicar el método sigamos...

>>m=(a+b)./2

m=0.25

es importante no olvidarnos de escribir ./ para que al MATLAB no trate a las variables en uso como insertas en operaciones matriciales

>>poli1(m)

f=0,003906

como nos da positivo el nuevo intervalo para aplicar el método debería ser

a=0,25

b=0,5

lo que podemos lograr si escribimos

>>a=m;

con las flechitas para arriba vamos a buscar la instrucción donde dividíamos por dos el nuevo intervalo y apretamos enter cuando lo encontramos

>>m=(a+b)/2

y automáticamente tenemos el nuevo punto medio

m=0.375

para ver el signo hacemos

>>poli1(m)

f=-0,480224

así que el próximo paso deberá ser

>>b=m

luego

>>m=(a+b)/2

nos arrojará

m=0.3125

>> poli1(m)

-0,240463

>>b=m;

luego

>>m=(a+b)/2

que nos mostrará que el método está convergiendo a una solución.

Siguiendo esta metodología se puede resolver los ejercicios de la práctica de Raíces de Ecuaciones.

Volviendo al tema de las funciones, se puede también crear una función de dos argumentos llamada medio(a,b) que permita encontrar $(a+b)/2$ en cada paso, que tendrá la siguiente redacción:

function f=medio(a,b)

f=(a+b)/2

De esta manera, estamos usando la computadora como si fuera una simple calculadora, pero a la que le empezamos a incorporar algunos aditamentos que el problema a resolver requiera

Sin embargo, siguiendo esa metodología de resolución, será el usuario quien, en cada paso, decida cuando reemplazar **m** por **a** y cuándo **m** por **b**. Si pretendemos que sea la computadora quien también realice esta tarea, vamos a necesitar elaborar un “archivo m” que sea no ya una función sino un **programa**

Pero antes vamos a tener que incorporar algunos elementos nuevos, que forman parte del lenguaje de programación, y que comenzaremos a delinear la próxima clase.

Clase 3.

UN BREVE REPASO.

En la primera clase vimos como introducir matrices y cómo hacer operaciones sencillas con las variables que cargamos en la memoria del MATLAB (y que además son todas consideradas como matrices). Vimos que cuando nos interese que las operaciones sean matriciales usaremos los operadores $+$, $*$, $./$, etc, pero cuando no nos interese que trabaje con matrices le anteponíamos un punto $./$, $.*$, $./$. Vimos también como graficar funciones, y como utilizar algunas funciones predefinidas por el programa como `humps`, `fzero`, `fmin`, etc.

En la segunda clase nos fuimos animando a un poco más, y aprendimos a crear nuestras propias funciones, y creamos la función `poli1` y la función `mitad` de uno y dos argumentos respectivamente.

Se comentó también que la creación de las funciones era uno de los usos de los llamados “archivos m”, y que las otras utilidades eran para guardar datos o para hacer programas.

Y, para no olvidarnos que lo que estamos aprendiendo del Matlab es para utilizarlo en Matemática Aplicada, *a través de la línea de comandos*, usamos las dos funciones que fuimos capaces de crear para encontrar los ceros de una función por el método de bisección.

Para algunos, seguramente habrá sido bastante novedoso el hecho de poder utilizar la computadora como una calculadora a la que le podíamos ir (muy de a poco) enseñando algunas cosas, y vimos que las iteraciones que hicimos para resolver por bisección la función `poli1(x)` fueron mucho menos laboriosas que si lo hubiéramos hecho con una calculadora.

Pero para otros, es probable que dieran por sentado que la máquina podía “aprender” lo que le enseñamos y destacaran, en cambio, que la máquina todavía no era capaz de ir “eligiendo” los intervalos que corresponden para poder “cercar” a la raíz después de cada iteración.

La cuestión es que esto también se lo vamos a poder “enseñar” a la máquina, pero para ello vamos a tener que *hacer un programa*, y para eso vamos a tener que aprender algunos términos nuevos, un nuevo léxico que es el que la máquina interpreta y constituye el llamado “lenguaje de programación” que utiliza el Matlab, que, por suerte, no es muy distinto al que pueden utilizar en otros lenguajes como el fortran, pascal, o basic.

INSTRUCCIONES DE PROGRAMACIÓN.

Vamos entonces a introducir algunas nuevas “instrucciones” que vamos a usar en los programas que hagamos de aquí en más.

Si bien son muchas las instrucciones de programación disponibles, veremos que con un pequeño puñado de ellas podremos realizar tareas muy complejas. Este pequeño puñado es el siguiente:

- a) Asignaciones (`=`).
- b) Ingresar de datos por teclado. (**input**)
- c) Mostrar resultados por pantalla (**disp**)
- d) Decidir entre distintas alternativas (**if... else ... end**)

- e) Repetir un grupo de instrucciones un número dado de veces (**for i=1:n ... end**)
- f) Repetir un grupo de instrucciones mientras se cumpla determinada condición (**while ... end**)

Volvemos a señalar que hay muchas más, pero, en el marco de la cursada, vamos a tratar de arreglarnos con este pequeño grupo, y se incorporarán las que resulten pertinentes en ocasión de la resolución.

Antes de entrar en detalles sobre el uso de estas instrucciones, volvamos a recordar que nos hemos planteado como objetivo desarrollar pequeños (o no tanto) programas, que lo que pretendemos que hagan es una determinada cantidad de tareas, en una secuencia determinada y un número determinado o no de veces.

Hasta ahora, hemos trabajado con el Matlab a través de la llamada Ventana de Comandos, en la cual hemos podido verificar lo obediente que resultó ser el utilitario, y a medida que dábamos una determinada instrucción, obteníamos en forma casi instantánea el resultado (en forma interactiva).

Ahora ya no nos va a interesar el resultado de cada instrucción que le demos, sino el resultado de *un conjunto de instrucciones* dadas en un cierto orden, así que para esto vamos nuevamente a trabajar ideando estas instrucciones fuera de esta ventana, y cuando consideremos que tenemos el paquete más o menos listo para largar, volveremos a la ventana de comandos del Matlab. Nuevamente acá vamos a trabajar en otro ámbito, que es el *editor* (el mismo que usamos para crear las funciones). Vimos que para acceder al editor propio del Matlab íbamos con el mouse pulsando en el menú desplegable **File**, **New** y **M-file** y ya estamos en otro ambiente donde las instrucciones que impartimos no tienen una respuesta inmediata, sino que cuando consideremos que las instrucciones que hemos encadenado están suficientemente pulidas y podamos esperar que el conjunto funcione, recién ahí la grabaremos (**File**, **Save As**) con un nombre determinado, y con ese nombre la llamaremos desde la línea de comandos

Vamos a los detalles...

a) Asignaciones

El signo **=** nos va a indicar que lo que está a la derecha del signo **=** pase a la variable cuyo nombre habremos escrito a su izquierda.

Así, **a=4*8** le indicará a la máquina que realice la operación $4*8$ y su resultado (es decir el número 32) pase a ser el contenido de una variable de la memoria llamada **a** (si no existía esa variable la cargará en la memoria con ese contenido en el instante que le demos la orden, y si ya existía con otro valor, se pierde el valor que tenía almacenado y pasa a contener un número 32).

También son asignaciones **a=4*b** (y en ese caso cargará en la variable **a**, lo que al momento de “ejecutar” la asignación contenga la variable de memoria **b** multiplicado por 4), **a=poli1(5)** (y en este caso cargará en la variable de memoria **a** el resultado de considerar que $x=5$ en la función que creamos la clase pasada), **a=a+1** (en este caso cargará en la variable **a** el resultado de lo que en el momento de ejecutar esa asignación contenía la variable **a** más 1, borrando el contenido anterior de la variable **a**, o sea que el resultado de esta asignación es incrementar en uno el contenido de la variable **a**)

b) Ingresar datos por teclado (instrucción **input**)

En la mayoría de los programas que se elaboren o utilicen en la cursada, habrá un grupo de datos que pretendemos que se carguen en la memoria de la máquina al principio de la ejecución de las operaciones que hayamos encadenado en el programa. Así por ejemplo, los valores **a** y **b**, que forman el extremo de un intervalo en el que vamos a aplicar el método de bisección, son parte de los ingresos que el programa deberá ingresar por teclado. También puede ser la cantidad de iteraciones que pretendemos que el programa ejecute, o la tolerancia que vamos a imponer para considerar que hemos llegado al resultado.

Normalmente son las primeras operaciones que nuestros programas van a ejecutar, y la instrucción es **input**.

Todas las instrucciones que usemos (el puñadito inicial propuesto) deberán respetar una cierta sintaxis porque sino el programa no las interpreta y seguramente viviremos recibiendo mensajes de error.

De las múltiples formas que el Matlab acepta para esta instrucción el siguiente ejemplo es el que nos muestra la forma en que nosotros vamos a usarla:

```
a=input('ingrese el extremo inferior del intervalo= ');
```

el resultado de esta instrucción es que aparecerá en la pantalla de la computadora el mensaje:

ingrese el extremo inferior del intervalo=

y al lado del signo igual veremos el cursor titilando a la espera que ingresemos un valor, (reitero que esto lo veremos sólo cuando hayamos decidido que el programa está listo y lo llamemos desde la línea de comandos), una vez pulsadas las teclas del número que querramos ingresar al pulsar *enter* el programa efectúa la *asignación* del ingreso del dato tipeado a la variable de memoria llamada **a** y continuará ejecutando la instrucción que hayamos escrito a continuación de este **input**.

En el siguiente ejemplo muestra **nuestro primer programa de computación!!!** que permite que se almacenen tres variables en la memoria de la máquina, **a**, **b** y **eadm** y además permite que se dibuje la función poli1 en el intervalo **a**, **b** y un circulito de color rojo en el valor medio del intervalo.

Este primer programa debe ser ingresado, al igual que cuando cargábamos una función, en un **archivo m**, cargado en el Editor del Matlab, seleccionando con el mouse las siguientes opciones del menú:

```
File
New
m-file
```

```
% programainput :ejemplo de utilización de la instrucción
input
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
eadm=input('ingrese el error admisible ');
hold on
```

```
fplot('poli1',[a,b])
grid
m=medio(a,b);
plot(m,poli1(m),'ro')
```

en el que el signo % es para que la máquina no lo tome como una orden a cumplir sino que es un comentario para que cuando volvamos a abrir el programa nos sirva para recordar su contenido.

Para que el programa funcione debemos grabarlo con un determinado nombre (por ejemplo : programainput), seleccionando con el mouse las siguientes opciones del menú:

File

Save As

Se tipea el nombre propuesto: programainput

Guardar

Y a partir de ese momento el programa quedará almacenado en la máquina para ser utilizado en forma inmediata en el modo interactivo (command window) simplemente tipeando el nombre dado al programa (y se ejecutarán todas las instrucciones almacenadas en ese archivo) o podrán ser utilizadas en otro momento, aunque en el interín hayamos debido apagar la computadora.

c) mostrar resultados por pantalla (instrucción disp)

Cuando lo que pretendamos sea que aparezca un mensaje en la pantalla, o que nos muestre el resultado de una variable de la memoria de la máquina, usaremos esta instrucción.

Normalmente la vamos a usar para mostrar los resultados de la ejecución del programa al final de la secuencia de instrucciones encadenadas, o en la mitad del programa cuando nos interese algún resultado parcial de la iteración.

Los siguientes ejemplos nos muestran posibles usos de la instrucción disp además de, fundamentalmente, mostrarnos la sintaxis que deberemos respetar para que funcione:

```
disp('mensaje textual a monitor')
```

```
disp('a=')
```

```
disp(a)
```

```
disp('poli1(a)=')
```

```
disp(poli1(a))
```

Para ver la utilización de esta instrucción en un programa podemos agregar algunas líneas al programa anterior que nos muestren algunos datos por pantalla, quedando el programa completo así:

```
% programainputdisp :ejemplo de utilización de la
instrucción input y disp
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
```

```

eadm=input('ingrese el error admisible ');
hold on
fplot('poli1',[a,b])
grid
m=medio(a,b);
plot(m,poli1(m),'ro')
disp('EL RESULTADO DE LA PRIMER ITERACIÓN ES m=');
disp(m)
disp('para el cual la función adopta un valor de ');
disp(poli1(m))

```

d) decidir entre distintas alternativas (instrucción if.. else... end)

Es bastante común que, en los diferentes métodos de resolución numérica de problemas, el “camino” de la resolución en algún momento se bifurque, y debemos decidir ejecutar una serie de tareas u otras según cuáles sean los valores que adopten algunas variables durante el proceso.

Un caso en el que podemos necesitar la instrucción if es para decidir si en el intervalo ingresado se puede utilizar el método de bisección o no.

```

% programaif1 :ejemplo de utilización de la instrucción if
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
if poli1(a).*poli1(b)<0
    disp('se puede aplicar el método en el intervalo
    ingresado')
else
    disp('no se puede aplicar el método en el intervalo
    ingresado')
end

```

Otro caso en el que resulta sumamente útil la instrucción if es para hallar las raíces de una ecuación por el Método de Bisección, ya que en cada paso de la iteración se divide el intervalo en dos, presentándose para el paso siguiente un subintervalo en el cual la función cambia de signo, y otro subintervalo en el cual el signo de la función es el mismo.

Si **a** y **b** son los extremos del intervalo y **m** es su punto medio (resultado de aplicar la función medio

m=medio(a,b)

entonces, para elegir el nuevo intervalo podremos usar:

if poli1(a).*poli1(m)<0

b=m;

else

a=m;

end

y el nuevo intervalo volverá a tener extremos denominados **a** y **b** pero será de la mitad de longitud del anterior y en él seguiremos teniendo asegurado el cambio de signo de la función.

Podemos agregar estas líneas a nuestro segundo programa (el que habíamos llamado programainputdisp) quedando en consecuencia así:

```
% programaif2 :ejemplo de utilización de la instrucción
input disp if
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
eadm=input('ingrese el error admisible ');
hold on
fplot('poli1',[a,b])
grid
m=medio(a,b);
plot(m,poli1(m),'ro')
disp('EL RESULTADO DE LA PRIMER ITERACIÓN ES m=');
disp(m)
disp('para el cual la función adopta un valor de ');
disp(poli1(m))
if poli1(a).*poli1(m) <0
    b=m;
else
    a=m;
end
disp('El nuevo subintervalo en el que continuaremos
aplicando el método es')
disp('a=');
disp(a)
disp('b=');
disp(b)
```

e) Repetir un grupo de instrucciones un número dado de veces (instrucción for i=1:n...end)

Para repetir un grupo de instrucciones una cantidad de veces prefijada puede utilizarse en los programas esta instrucción. El resultado de la ejecución del siguiente programa será la aparición en pantalla de los números 1 2 3 4 5 6 7 8 9 y 10:

```
%p6 : ejemplo de utilización de la instrucción for...
n=10;
for i=1:n
    disp(i)
end
```

Otra variante en la sintaxis de esta instrucción es la siguiente:

```
%p7 : otro ejemplo de utilización de la instrucción for...
n=10;
for i=1:0.05:n
    disp(i)
end
```

Y, como resultado de la ejecución de este programa veremos en pantalla los números 1 1.05 1.10 1.15 ... y así siguiendo hasta llegar al número 10

En el siguiente ejemplo se generaliza un poco más la aplicación de la instrucción:

```
%p8 otro ejemplo de la instruccion for
vi=input('valor inicial=');
vf=input('valor final=');
h=input('incremento=');
for j=vi:h:vf
    disp(j);
    disp(j.*j);
end
```

Si incorporamos esta instrucción al programa en elaboración sobre el método de bisección para indicarle la cantidad de iteraciones a realizar hasta “cercar” a la raíz, nuestro programa quedará como sigue:

```
% biseccion :ejemplo de utilización de la instrucción input
disp if for
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
eadm=input('ingrese el error admisible ');
hold on
fplot('poli1',[a,b])
grid
cantidad_iteraciones=input('ingrese cuantas iteraciones
desea realizar =')
for i=1:cantidad_iteraciones
    m=medio(a,b);
    plot(m,poli1(m),'ro')
    disp('EL RESULTADO DE LA ITERACIÓN ES m=');
    disp(m)
    disp('para el cual la función adopta un valor de ');
    disp(poli1(m))
    if poli1(a).*poli1(m) <0
        b=m;
    else
        a=m;
    end
    disp('El nuevo subintervalo en el que continuaremos
aplicando el método es')
    disp('a=');
    disp(a)
    disp('b=');
    disp(b)
    pause
end
```

la instrucción pause, incluida en esta versión del programa, interrumpe la ejecución del programa hasta que se pulse cualquier carácter del teclado.

f) Repetir un grupo de instrucciones mientras se cumpla determinada condición (while ... end)

En algunas ocasiones el número de veces que se repetirán las instrucciones no está prefijado de antemano, sino que surge de, por ejemplo, la evolución que tengan los contenidos de distintas variables de memoria. Un claro ejemplo de esta situación sería la cantidad de iteraciones a realizar en un determinado método, que puede depender de los valores iniciales, del tipo de función, de la velocidad de convergencia, de los límites de tolerancia impuestos, etc.

En estas situaciones resulta de suma utilidad esta instrucción

El siguiente ejemplo irá cambiando el contenido de la variable *j* desde el valor inicial que le demos hasta que lleguemos al valor máximo que ingresemos:

```
%p8 primer ejemplo de la instrucción while
j=input('valor inicial=');
tope=input('máximo valor al que desea llegar=');
while j<tope
    j=j+1
end
```

En el siguiente ejemplo se ingresa un valor “adentro” del lazo delimitado por la instrucción *while*, y la ejecución continuará mientras se cumpla la condición indicada (*j<tope*)

```
%p9 segundo ejemplo de la instrucción while
j=input('valor inicial=');
tope=input('máximo valor al que desea llegar=');
while j<tope
    j=input('valor de control')
end
```

En el siguiente ejemplo se ejecutarán las instrucciones que coloquemos entre *while* y *end* mientras se cumpla que el valor de la función en el punto medio entre los valores *a* y *b* se mantenga mayor que la tolerancia que hayamos especificado en la variable **eadm**

```
% wbiaseccion :ejemplo de utilización de la instrucción
input disp if for y while
clear
hold off
a=input('ingrese el extremo inferior del intervalo ');
b=input('ingrese el extremo superior del intervalo ');
eadm=input('ingrese el error admisible ');
hold on
fplot('poli1',[a,b])
grid
m=medio(a,b);
while (abs(poli1(m))>eadm)
    m=medio(a,b);
```

```

plot(m, poli1(m), 'ro')
disp('EL RESULTADO DE LA ITERACIÓN ES m=');
disp(m)
disp('para el cual la función adopta un valor de ');
disp(poli1(m))
if poli1(a).*poli1(m) < 0
    b=m;
else
    a=m;
end
disp('El nuevo subintervalo en el que continuaremos
aplicando el método es')
disp('a=');
disp(a)
disp('b=');
disp(b)
pause
end
disp('para el error admitido la raíz calculada es m=')
disp(m)

```

Durante el transcurso de las clases se desarrollarán otros programas que permitan familiarizarse con estas instrucciones y combinarlas de manera de poder cumplir con el propósito de resolver situaciones complejas para las que la línea de comandos resulta insuficiente, y también se utilizarán programas ya elaborados que permitan la resolución de problemas con los métodos incluidos en el programa de la Materia.